

การเก็บข้อมูลแบบถาวรบน Cloud Storage ด้วย Flutter

By Prasertsak U., Ph.D

Outlines

- การจัดรูปแบบข้อมูลและบันทึกข้อมูลในรูปแบบ JSON
- การเปิดใช้งาน Cloud Storage ด้วย Google Firebase
- การบันทึก และการอ่านข้อมูลจากแหล่งบันทึกข้อมูลบนคลาวด์

What is JSON format ?

- The JSON format is text-based and is independent of programming languages.
- JSON uses the **key/value** pair, and the key is enclosed in quotation marks followed by a colon and then the value like **"id": "100"**. You **use a comma (,) to separate multiple key/value pairs**.

KEY	COLON	VALUE
"id"	:	"100"
"quantity"	:	3
"in_stock"	:	true

Objects and Arrays

- Objects are declared by curly ({}) brackets
- Arrays are declared by the square ([]) brackets

TYPE	SAMPLE
Object	<pre>{ "id": "100", "name": "Vacation" }</pre>
Array with values only	<pre>["Family", "Friends", "Fun"]</pre>

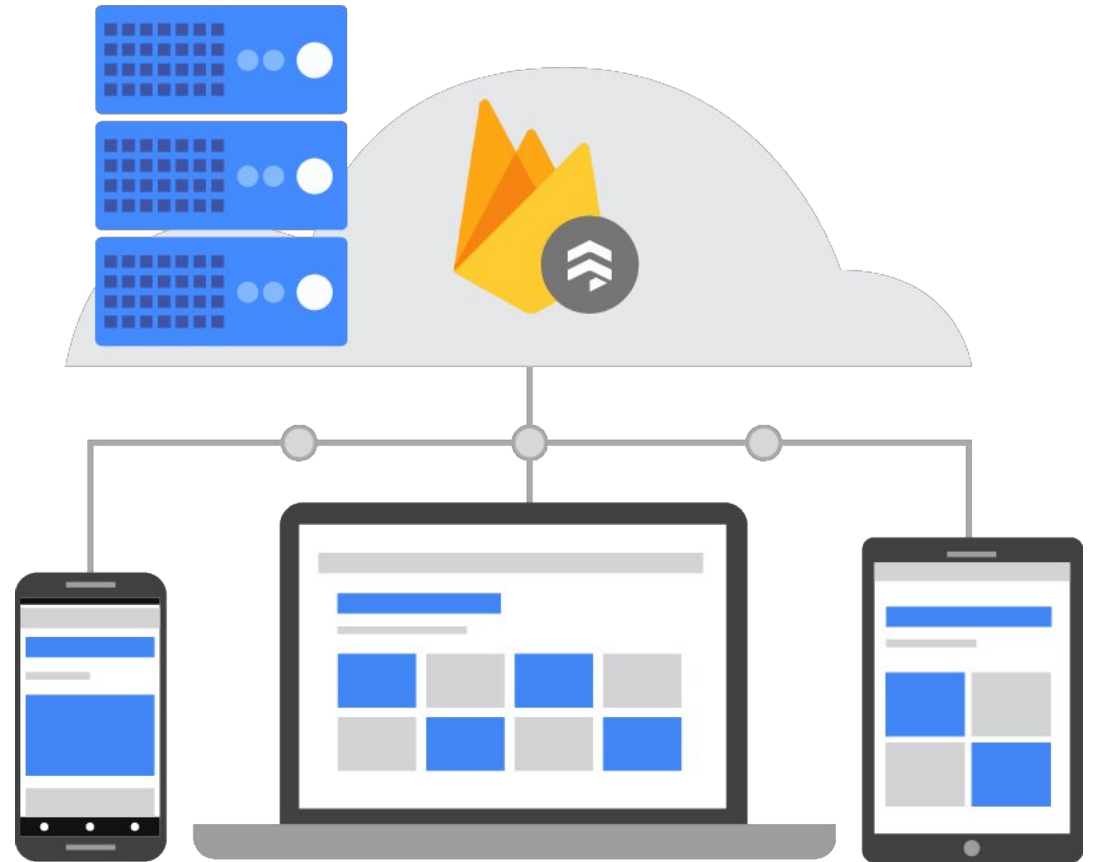
ตัวอย่างรูปแบบ JSON

TYPE	SAMPLE
Array with key/value	<pre>[{ "id": "100", "name": "Vacation" }, { "id": "102", "name": "Birthday" }]</pre>
Object with array	<pre>{ "id": "100", "name": "Vacation", "tags": ["Family", "Friends", "Fun"] }</pre>

TYPE	SAMPLE
Multiple objects with arrays	<pre>{ "journals": [{ "id": "4710827", "mood": "Happy" }, { "id": "427836", "mood": "Surprised" }], "tags": [{ "id": "100", "name": "Family" }, { "id": "102", "name": "Friends" }] }</pre>

Using Google Cloud Firestore

- Google Cloud Firestore is a flexible, scalable, [cloud-native database](#) for mobile, web, and server development.



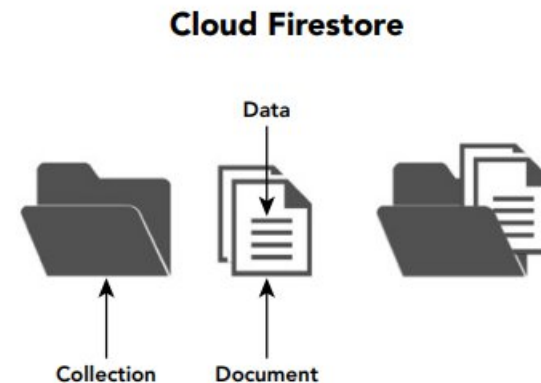
Data Structure comparison

- SQL Database vs Firebase - Cloud Firestore

SQL SERVER DATABASE	CLOUD FIRESTORE
Table	Collection
Row	Document
Columns	Data

In Cloud Firestore, a collection can contain only documents. A document is a key-value pair.

Documents cannot point to another document and must be stored in collections



A Sample Data

TYPE	VALUE
Collection	journals
Document	R5NcTWAaWtHTttYtPoOd
Document data as key-value pair	date: "2019-0202T13:41:12.537285" mood: "Happy" note: "Great movie." uid: "F1GGeKiwp3jRpoCVskdBNmO4GUN4"

```
{
  "journals": [
    {
      "R5NcTWAaWtHTttYtPoOd1": {
        "date": "2019-0202T13:41:12.537285",
        "mood": "Happy",
        "note": "Great movie.",
        "uid": "F1GGeKiwp3jRpoCVskdBNmO4GUN4",
      }
    }
  ]
}
```

🏠 > journals > R5NcTWAaWtH...		
📁 journal-3ced1	📁 journals	📄 R5NcTWAaWtHTttYtPoOd
+ Add collection	+ Add document	+ Add collection
journals >	R5NcTWAaWtHTttYtPoOd >	+ Add field
		date: "2019-02-02T13:41:12.537285"
		mood: "Happy"
		note: "Great movie."
		uid: "F1GGeKiwp3jRpoCVskdBNmO4GUN4"

ติดตั้งเครื่องมือเพิ่มเติมสำหรับการจัดการ Firebase

- เปิดใช้งาน Windows Command หรือ Terminal
- ติดตั้ง Firebase CLI

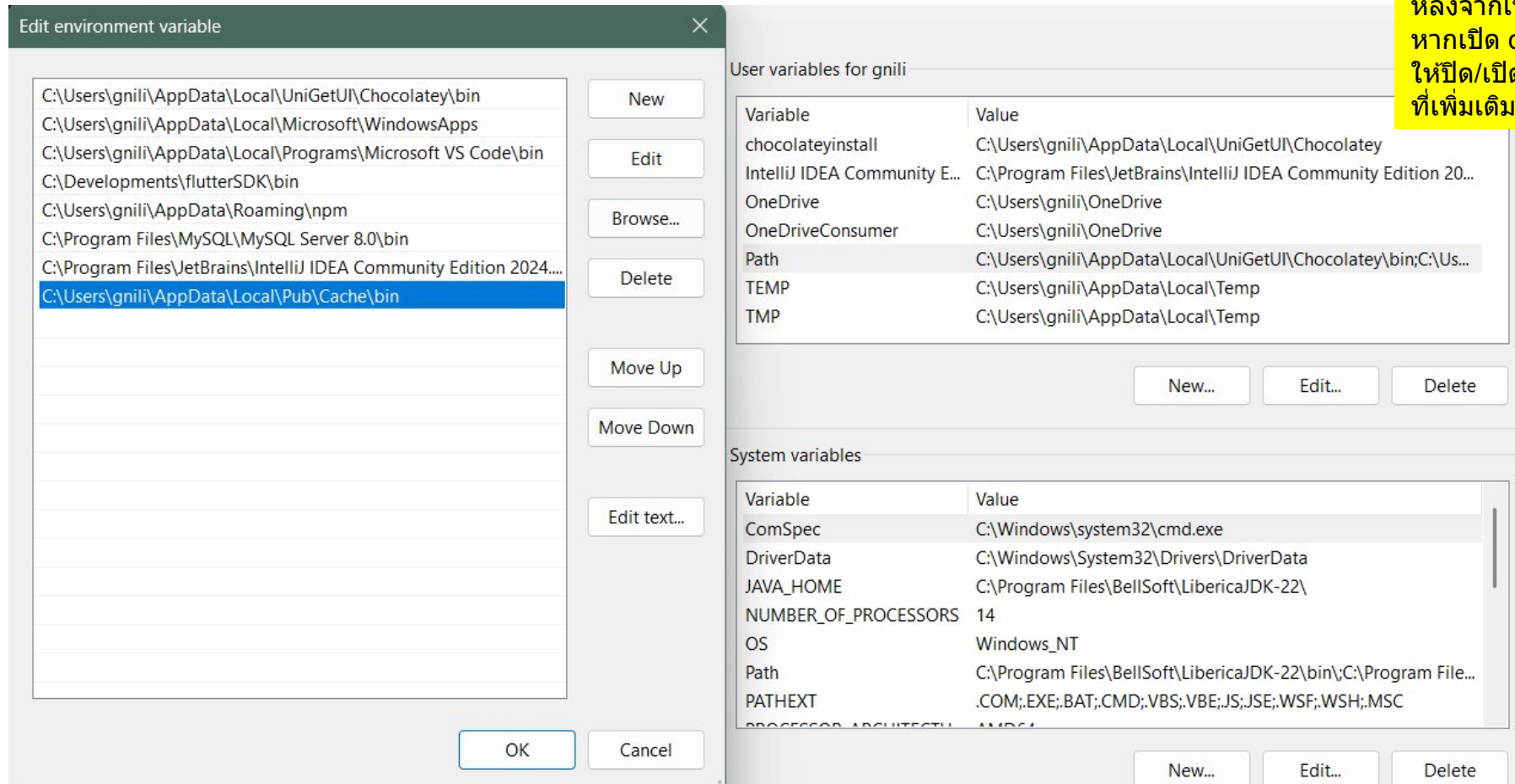
```
npm install -g firebase-tools
```

- ติดตั้ง FlutterFire CLI

```
dart pub global activate flutterfire_cli
```

Add Path for Flutterfire CLI for Windows

- Open Edit the System Environment Variables and add new path



หลังจากเพิ่มเติม path เสร็จเรียบร้อยแล้ว
หากเปิด command prompt ทิ้งไว้
ให้ปิด/เปิดใหม่ จึงจะสามารถใช้ path
ที่เพิ่มเติมใหม่ได้

Add Path for **Flutterfire** CLI **for MacOS**

สำหรับ MacOS ที่ใช้ bash
ให้เปลี่ยนชื่อไฟล์เป็น **.bashrc**
แทน **.zshrc**

- เปิดโปรแกรม Terminal และใช้คำสั่ง nano หรือ vi ในการเปิดไฟล์ **.zshrc**
nano ~/.zshrc
- ทำการเพิ่มคำสั่ง **export PATH="\$PATH": "\$HOME/.pub-cache/bin"**
ไปยังบรรทัดไฟล์ **.zshrc**



```
UW PICO 5.09 File: /Users/tinygrain/.zshrc

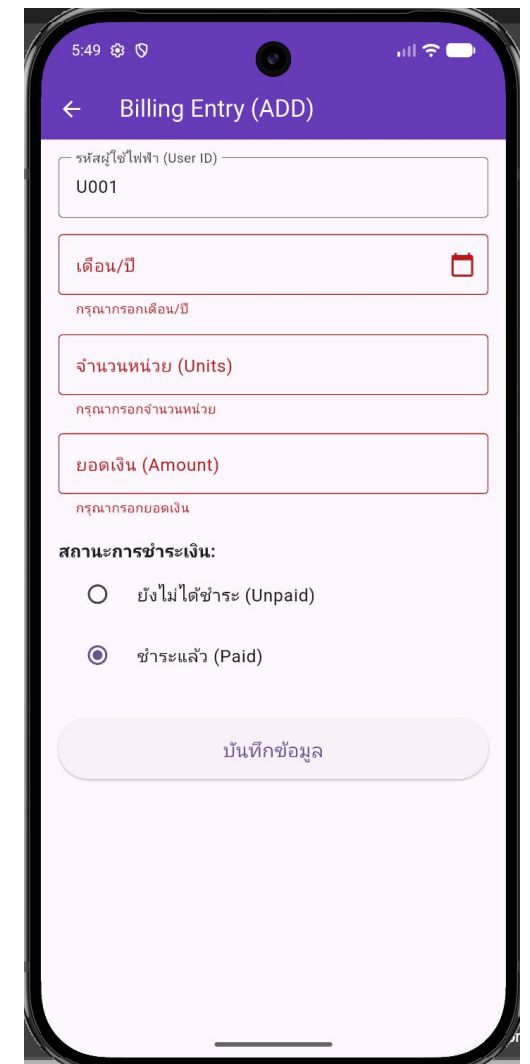
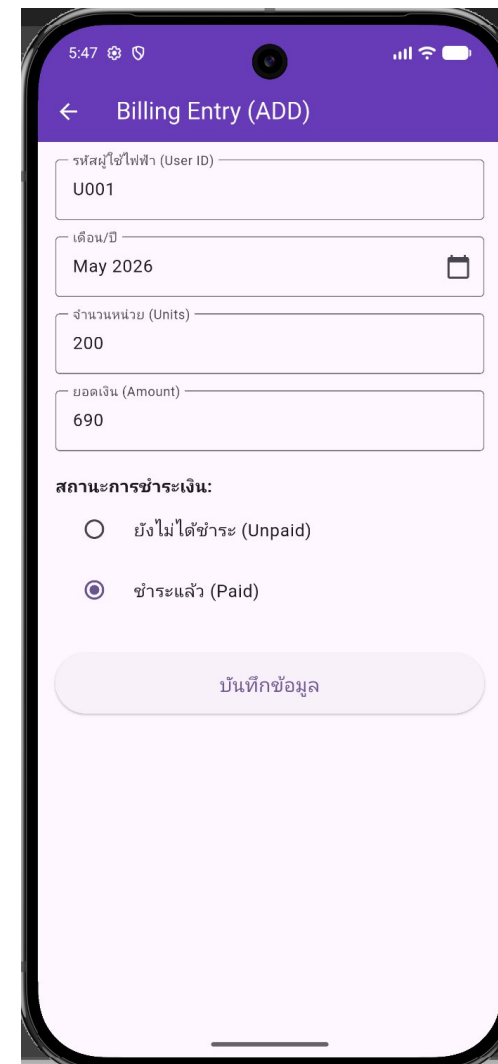
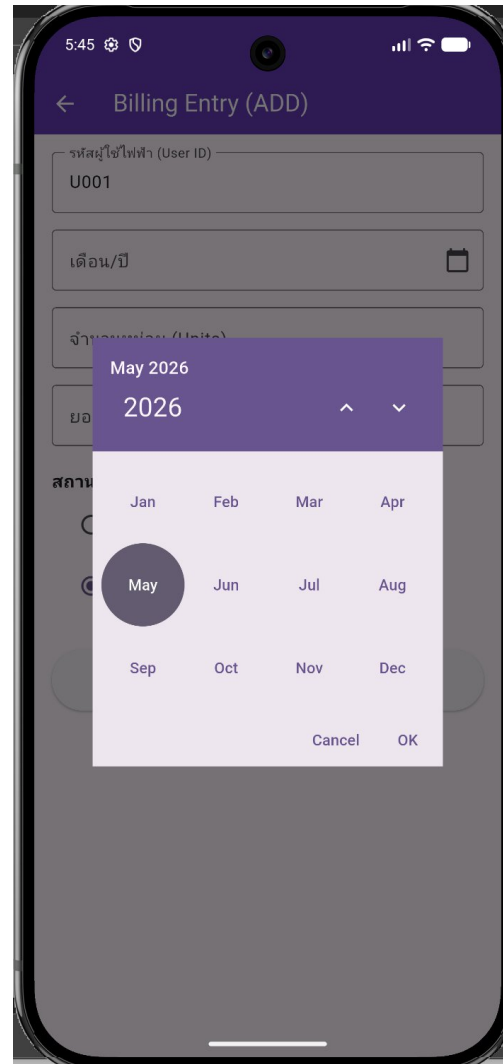
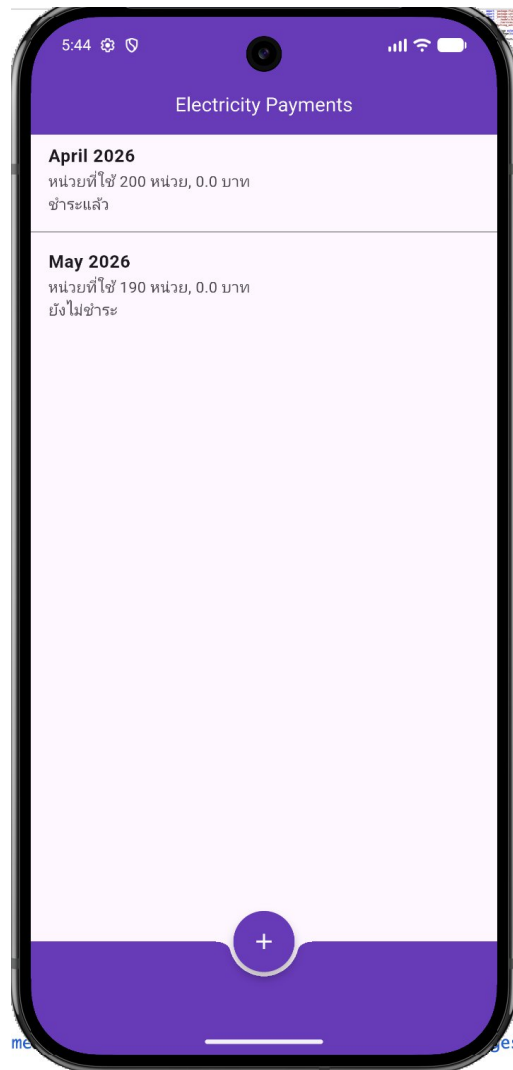
export PATH=$PATH:/Users/tinygrain/Development/flutterSDK/bin

export PATH="/opt/homebrew/opt/openjdk@21/bin:$PATH"
export CPPFLAGS="-I/opt/homebrew/opt/openjdk@21/include"
export JAVA_HOME=$(/usr/libexec/java_home -v 21)
export PATH="$PATH": "$HOME/.pub-cache/bin"
```

หลังจากแก้ไขและบันทึกเรียบร้อยแล้วให้ใช้คำสั่งต่อไปนี้เพื่อใช้งาน
source ~/.zshrc

เมื่อเพิ่มเติมแล้วให้กด ctrl+x ตอบ Y และกด Enter เพื่อบันทึก

Building an ElectricityPayment App



Building an ElectricityPayment App

- สร้าง Flutter project ใหม่ชื่อว่า *electricity_payment*
- Download โครงสร้างตัวอย่างเบื้องต้นสำหรับใช้งาน [ที่นี่](#)
- นำ folder: *lib* ทั้งหมดวางทับใน Project: **electricity_payment**
- เปิดด้วย VS Code พร้อมพัฒนาต่อไป

Install Dependencies Library

- Install dependencies on '*pubspec.yaml*'

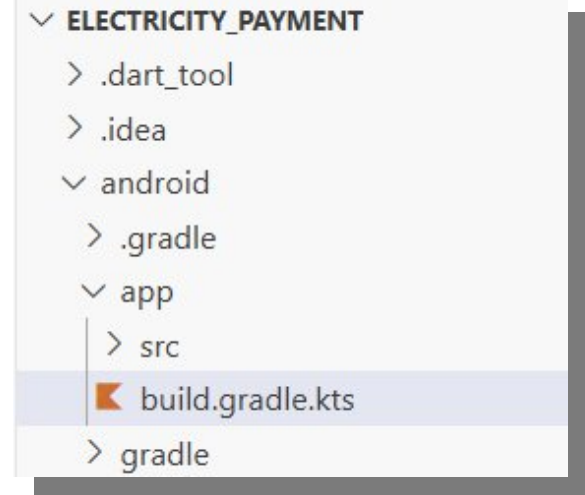
```
30 dependencies:
31   flutter:
32     sdk: flutter
33
34   # The following adds the Cupertino Icons font to your application.
35   # Use with the CupertinoIcons class for iOS style icons.
36   cupertino_icons: ^1.0.8
37   cloud_firestore: ^6.3.0
38   firebase_core: ^4.7.0
39   intl: ^0.20.2
40   month_picker_dialog: ^6.7.2
41
```

หมายเหตุ: ตัวเลขเวอร์ชันอาจมีการเปลี่ยนแปลงไปตามการอัปเดตของ Firebase SDK และ Flutter สามารถตรวจสอบเวอร์ชันล่าสุดได้จาก pub.dev

Register App on Firebase

- Android - Open file : [android/app/build.gradle.kts](#)
- Goto **defaultConfig** and copy *applicationId* for registration ("com.example.electricity_payment")

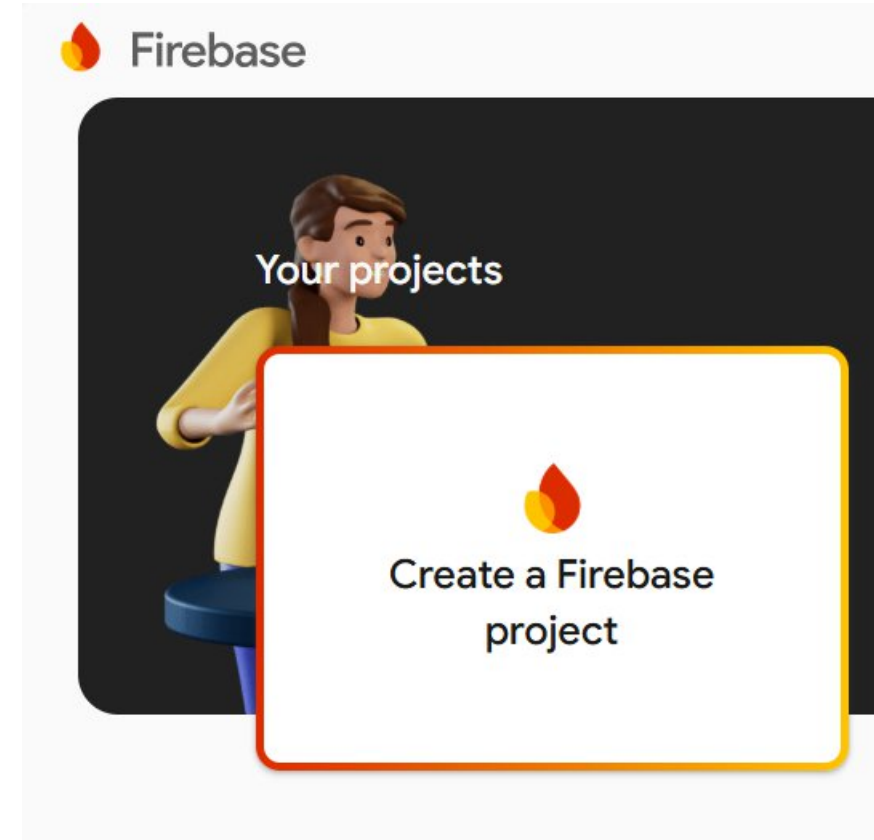
```
28 .....defaultConfig{
29 .....    // TODO: Specify your own unique Application ID (http
        .....    html).
30 .....    applicationId = "com.example.electricity_payment"
31 .....    // You can update the following values to match your
32 .....    // For more information, see: https://flutter.dev/to/
33 .....    minSdk = flutter.minSdkVersion
34 .....    targetSdk = flutter.targetSdkVersion
35 .....    versionCode = flutter.versionCode
36 .....    versionName = flutter.versionName
37 .....}
```



copy applicationId
เก็บไว้เพื่อนำไป
register กับ firebase

Firestore Configuration

- Go to Firestore Console:
<https://console.firebase.google.com/>
- Login and Create New Project for our App



Create Firebase Project

×

Create a project

Let's start with a name for your project[?]

Project name

FlutterFirebaseEducated

flutterfirebaseeducated

☒ I accept the [Firebase terms](#)

☐ Join the [Google Developer Program](#) to enrich your developer journey with access to AI assistance, learning resources, profile badges, and more!

Already have a Google Cloud project?
[Add Firebase to Google Cloud project](#)

Continue

×

Create a project

Gemini in Firebase is integrated within the Firebase console to help streamline your development process.

- ◆ Chat with Gemini to plan and design your application, troubleshoot issues, and get recommendations tailored to your project
- ⚙️ Debug and troubleshoot issues in your apps using AI assistance in Firebase Crashlytics
- 📧 Get campaign insights and recommendations to engage your users through Firebase Cloud Messaging
- 🔗 Generate schema and explore your data using natural language in Firebase Data Connect

☒ Enable Gemini in Firebase
Recommended

Disclaimers:

Gemini in Firebase is still under development and may give inaccurate information. Test any code it generates before using it in your apps. Gemini can interact with your Firebase data and tailor response to your app, and it may use your prompts or data for training. [Learn more about how we train our models](#) Use of Gemini in Firebase is subject to [Google Terms of Service](#) and the [Generative AI Prohibited Use Policy](#)

Previous

Continue

Create Firebase Project (cont.)

× Create a project

Google Analytics for your Firebase project

Google Analytics is a free and unlimited analytics solution that enables targeting, reporting, and more in Firebase Crashlytics, Cloud Messaging, In-App Messaging, Remote Config, A/B Testing, and Cloud Functions.

Google Analytics enables:

- × A/B testing ?
- × User segmentation & targeting across Firebase products ?
- × Breadcrumb logs in Crashlytics ?
- × Event-based Cloud Functions triggers ?
- × Free unlimited reporting ?

☐ Enable Google Analytics for this project
Recommended

[Previous](#)

Create project



Preparing your project, please wait.

FlutterFirebaseEducated

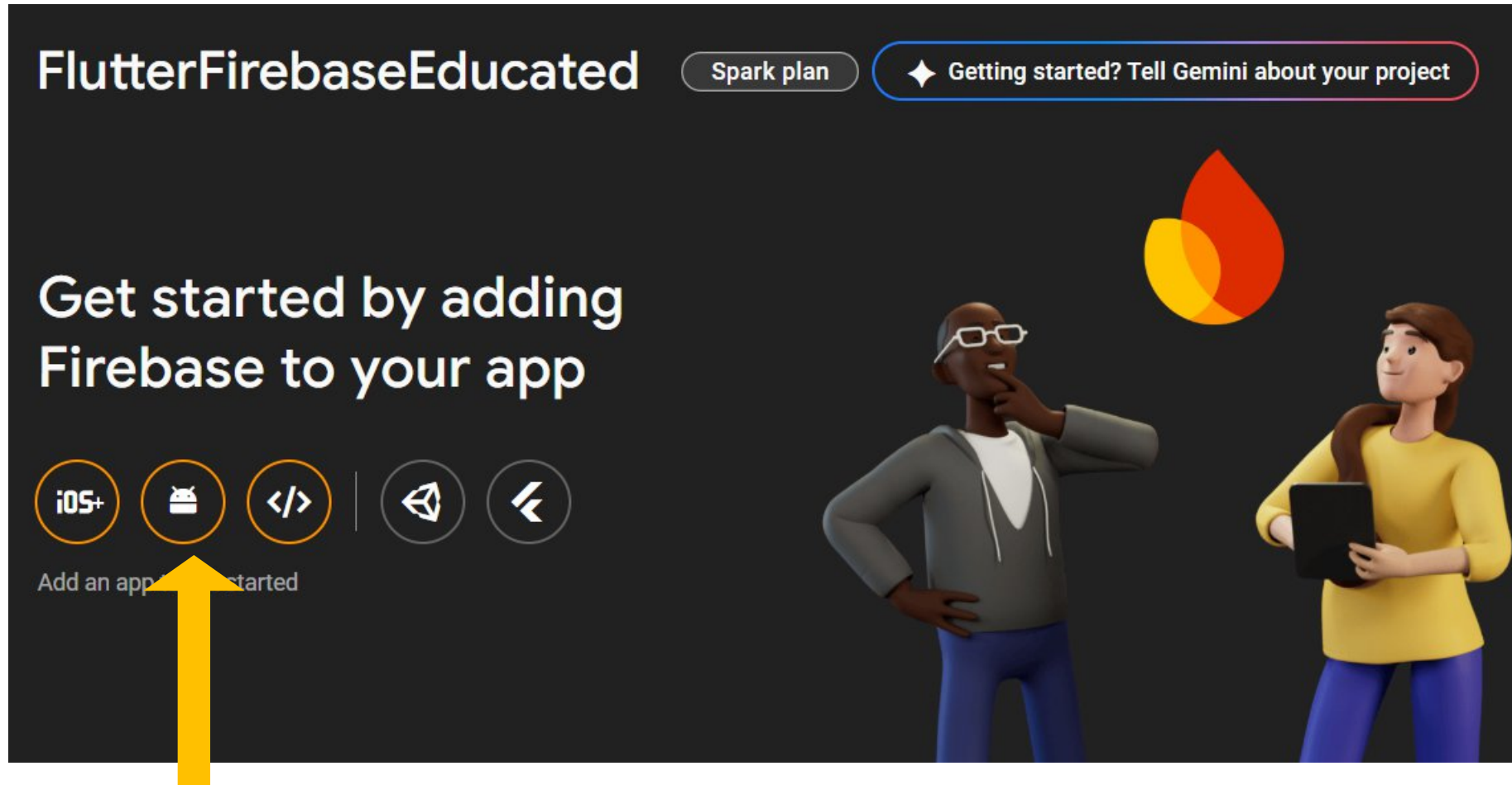


FlutterFirebaseEducated

✓ Your Firebase project is ready

Continue

Registering Android App



เลือก

Registering Android App

[illegible]

×

Add Firebase to your Android app

✓

Register app

Android package name: com.example.test_firebase

2

Download config file

Download google-services.json

Switch to the **Project** view in Android Studio to see your project root directory.

Move the google-services.json file you just downloaded into your Android app module root directory.

google-services.json

Next

3. Download file

Folder: android/app

Project Packages Scratchpad

MyApplication (~ / Desktop / My

▶ .gradle

▶ .idea

▼ app

▶ build

▶ libs

▶ src

▶ .gitignore

▶ app.iml

▶ build.gradle

▶ google-services.json

▶ proguard-rules.pro

▶ gradle

4. กดปุ่ม Next

Continue to config flutter project

- เปิดไฟล์ `android/app/build.gradle.kts`
- เพิ่มบรรทัดตามคำแนะนำต่อไปนี้ในส่วนของ plugins

```
1  plugins {  
2      id("com.android.application")  
3      id("kotlin-android")  
4      // The Flutter Gradle Plugin must be applied after the Android  
5      id("dev.flutter.flutter-gradle-plugin")  
6      id("com.google.gms.google-services") ← ----  
7  }  
8  
9  dependencies {  
10     // Import the Firebase BoM  
11     implementation(platform("com.google.firebase:firebase-bom:34.1.0"))  
12 }
```

เพิ่มบรรทัดที่ 6 นี้

Module (app-level) Gradle file (<project>/<app-module>/build.gradle.kts):

```
plugins {  
    id("com.android.application")  
    // Add the Google services Gradle plugin  
    id("com.google.gms.google-services")  
    ...  
}  
  
dependencies {  
    // Import the Firebase BoM  
    implementation(platform("com.google.firebase:firebase-bom:34.1.0"))  
  
    // TODO: Add the dependencies for Firebase products you want to use  
    // When using the BoM, don't specify versions in Firebase dependencies  
    // https://firebase.google.com/docs/android/setup#available-libraries  
}
```


แก้ปัญหาการ Build Error ที่อาจจะเกิดขึ้น

- เมื่อเป้าหมายในการรันไม่ใช่ Web Browser แต่เป็นอุปกรณ์ Android หรือ Android Emulator การ Build ด้วยสภาพแวดล้อมในปัจจุบันทั้ง Version ของ FlutterSDK และ Version ของ Plugin ที่ใช้ จะแสดง Error ดังตัวอย่าง

```
Your project is configured with Android NDK 26.3.11579264, but the following plugin(s) depend on a different Android NDK version:  
- cloud_firestore requires Android NDK 27.0.12077973  
- firebase_core requires Android NDK 27.0.12077973  
Fix this issue by using the highest Android NDK version (they are backward compatible).  
Add the following to C:\Users\gnili\Documents\02739342\Flutter-3.29.3\Lab-Projects\electricity_payment\android\app\build.gradle.kts:
```

```
android {  
    ndkVersion = "27.0.12077973"  
    ...  
}
```

```
Suggestion: use a compatible library with a minSdk of at most 21,  
            or increase this project's minSdk version to at least 23,  
            or use tools:overrideLibrary="io.flutter.plugins.firebase.firestore" to force usage (may lead to runtime failures)
```

Continue to config flutter project (cont.)

- เปิดไฟล์ `android/local.properties`
- เพิ่มเติมตัวแปร `flutter.ndkVersion` และ `flutter.minSdkVersion` ดังนี้

```
3 flutter.buildMode=debug
4 flutter.versionName=1.0.0
5 flutter.versionCode=1
6 flutter.ndkVersion=27.0.12077973
7 flutter.minSdkVersion=23
8 |
```

ตัวแปรทั้ง 2 ตัวนี้จะถูกนำไปใช้ในไฟล์
`android/app/build.gradle.kts`

- หากเพิ่มแล้วยังเกิดปัญหาเดิม ให้เข้าไปกำหนด version ของตัวแปรนั้นโดยตรงภายในไฟล์
`android/app/build.gradle.kts`

```
14 android {
15     namespace = "com.example.electricity_payment"
16     compileSdk = flutter.compileSdkVersion
17     ndkVersion = "27.0.12077973" ←-----
```

```
28 ... defaultConfig {
29     ... // TODO: Specify your own unique Application ID (ht
30     ... applicationId = "com.example.electricity_payment"
31     ... // You can update the following values to match you
32     ... // For more information, see: https://flutter.dev/t
33     ... minSdk = 23 ←-----
34     ... targetSdk = flutter.targetSdkVersion
35     ... versionCode = flutter.versionCode
36     ... versionName = flutter.versionName
37 }
```

แก้ปัญหาคำ Build Error ที่อาจจะเกิดขึ้น

- หากเกิด error ในขณะที่คอมไพล์เพื่อรันบน Android ในลักษณะนี้ ซึ่งเกี่ยวกับ flutter.versionCode ภายในไฟล์ *android/app/build.gradle.kts*

flutterVersionCode must be an integer.

- ให้เข้าไปแก้ไขไฟล์ *android/app/build.gradle.kts*

```
28 minSdk = flutter.minSdkVersion
29 targetSdk = flutter.targetSdkVersion
30 //versionCode = flutter.versionCode
31 versionCode = project.findProperty("flutter.versionCode")?.toString()?.toIntOrNull() ?: 1
32 versionName = flutter.versionName
```

ยกเลิกด้วยการใส่ comment

↑
เพิ่มทดแทนของเดิม

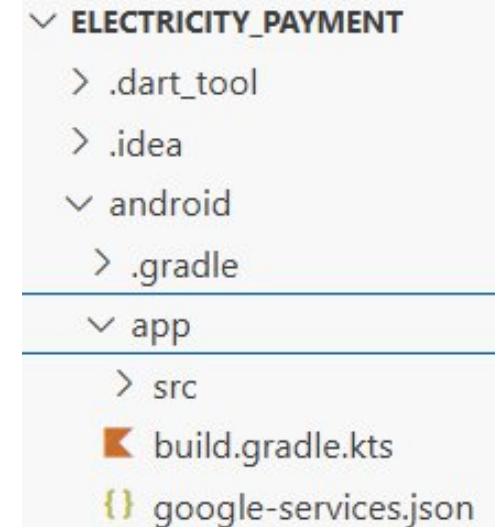
ติดตั้งไฟล์ google-services.json

ณ ปัจจุบันขอข้ามขั้นตอนนี้ไปก่อน
เนื่องจากการใช้เครื่องมืออื่นทดแทน
อาจทำให้ไม่จำเป็นต้องใช้วิธีการนี้

- ทำการ copy ไฟล์ *google-services.json* ที่ได้ download ในขั้นตอนก่อนหน้านี้
- เข้าไปวางไว้ใน folder: *android->app*

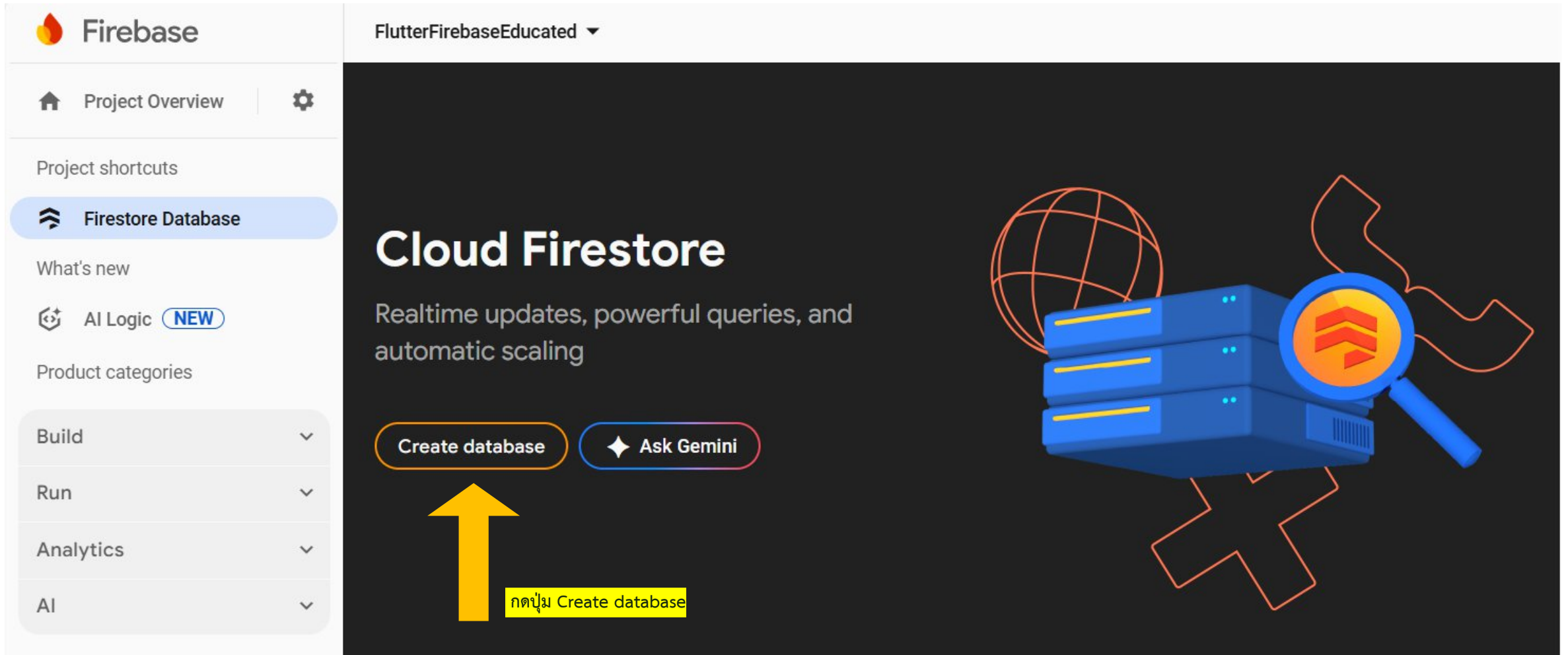
google-services.json

```
7  "client": [  
8      {  
9          "client_info": {  
10             "mobilesdk_app_id": "1:397271401538:android:f62cbbe1b35c2d299859db",  
11             "android_client_info": {  
12                 "package_name": "com.example.electricity_payment"  
13             }  
14         },  
15     ]
```



```
▼ ELECTRICITY_PAYMENT  
  > .dart_tool  
  > .idea  
  ▼ android  
    > .gradle  
  ▼ app  
    > src  
    build.gradle.kts  
    google-services.json
```

Create Firestore database



The image shows the Firebase console interface for a project named 'FlutterFirebaseEducated'. The left sidebar contains the following elements:

- Project Overview (with a gear icon)
- Project shortcuts
 - Firestore Database** (highlighted)
- What's new
 - AI Logic (marked as NEW)
- Product categories
 - Build
 - Run
 - Analytics
 - AI

The main content area displays 'Cloud Firestore' with the description 'Realtime updates, powerful queries, and automatic scaling'. Below this, there are two buttons: 'Create database' and 'Ask Gemini'. A large yellow arrow points to the 'Create database' button, and a yellow box with the Thai text 'กดปุ่ม Create database' (Press the Create database button) is positioned next to the arrow.

Create Firestore database

Create database

1 Set name and location

2 Secure rules

★ Interested in Firestore with MongoDB compatibility? Create a Firestore Enterprise database in the Google Cloud Console

Learn more

Database ID

(default)

Location

asia-southeast1 (Singapore)

เลือก ตำแหน่งของ server ที่ต้องการสร้าง Database

ⓘ Your location setting is where your Cloud Firestore data will be stored

⚠ After you set this location, you cannot change it later.

Learn more

Cancel

Next

Create Firestore database

Create database

1 Set name and location

2 Secure rules

After you define your data structure, you will need to write rules to secure your data.
[Learn more](#)

☐ Start in **production mode**

Your data is private by default. Client read/write access will only be granted as specified by your security rules.

1. เลือก Test mode เพื่อการทดสอบ

☒ Start in **test mode**

Your data is open by default to enable quick setup. However, you must update your security rules within 30 days to enable long-term client read/write access.

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if
        request.time < timestamp.date(2025, 9, 12);
    }
  }
}
```

!

The default security rules for test mode allow anyone with your database reference to view, edit and delete all data in your database for the next 30 days

Cancel

Create

ใน Test Mode จะกำหนดวันสิ้นสุดเอาไว้ด้วย หากเลยวันที่กำหนดจะใช้งาน Firestore ไม่ได้ (แต่ข้อมูลยังอยู่) สามารถเข้ามาแก้ไขวันที่ได้

2. กดปุ่ม Create

เชื่อมต่อ Flutter Project กับ Firebase

- รันคำสั่งนี้ในรูทไดเรกทอรีของโปรเจกต์ เพื่อให้ FlutterFire CLI ตั้งค่าไฟล์ [FirebaseOption.dart](#) ในการกำหนดค่าต่าง ๆ ของ Firebase โดยอัตโนมัติ
- เปิด Terminal ภายใน VS-Code และรันคำสั่งนี้

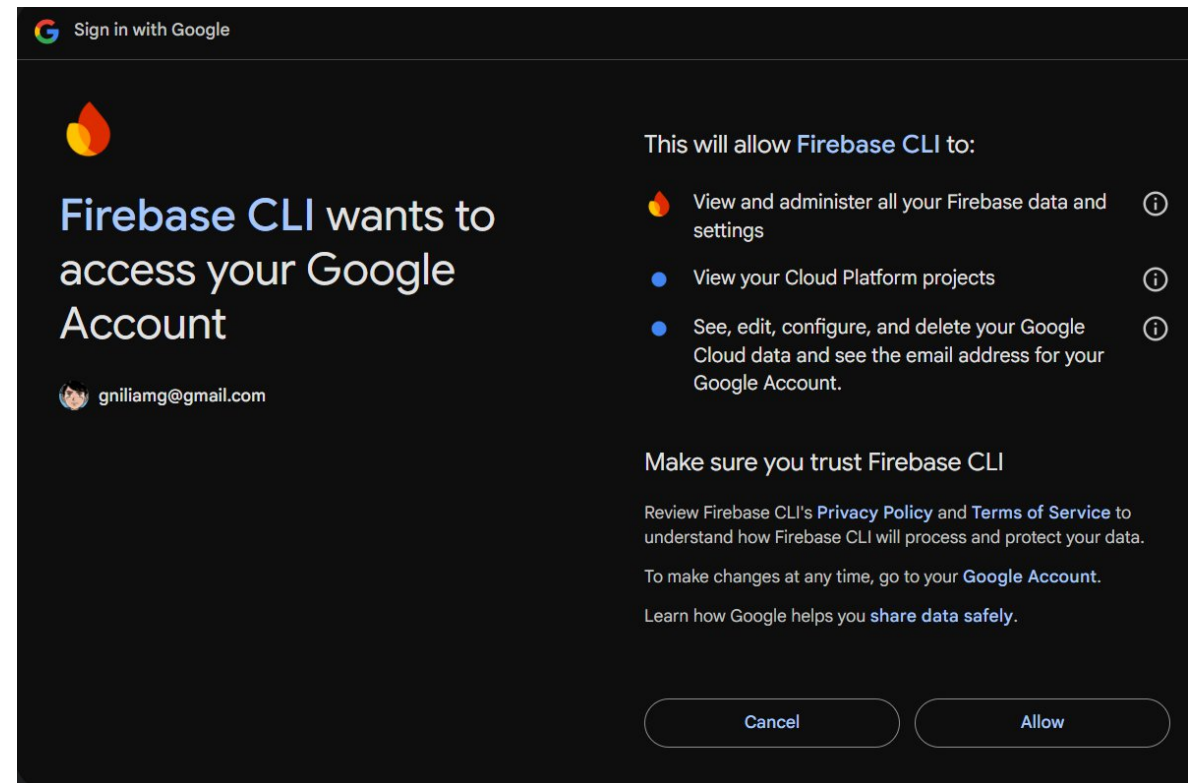
```
firebase login
```

ดำเนินการ Login ด้วย google account
ที่ทำการสร้าง Firebase Project ให้เรียบร้อย
และกด Allow เพื่ออนุญาต

Woohoo!

Firebase CLI Login Successful

You are logged in to the Firebase Command-Line interface. You can immediately close this window and continue using the CLI.



เชื่อมต่อ Flutter Project กับ Firebase

- หลังจากดำเนินการคำสั่ง firebase login เสร็จเรียบร้อยแล้ว ใน Terminal ภายใต้ VS-Code ให้รันคำสั่งต่อไปนี้

```
flutterfire configure
```

ให้เลือกชื่อ Project ที่ต้องการเชื่อม ด้วยการเลื่อนปุ่มลูกศร และกด Enter เมื่อต้องการ กดเลือก Android (ให้กดที่ **spacebar** สำหรับการเลือกหรือไม่เลือก) และรอจนแล้วเสร็จ (หลังเสร็จสิ้นจะเกิดไฟล์ชื่อ `firebase_options.dart` ภายใน lib)

```
tinygrain@MacBook-Air-khxng-TinyGrain electricity_payment % flutterfire configure
```

```
i Found 3 Firebase projects.
```

```
? Select a Firebase project to configure your Flutter application with >
```

```
> flutterfirebaseeducated (FlutterFirebaseEducated)
```

```
ict-angular-project (ict-angular-project)
```

```
iotlabprojects-76897 (IoTLABProjects)
```

```
<create a new project>
```

```
tinygrain@MacBook-Air-khxng-TinyGrain electricity_payment % flutterfire configure
```

```
i Found 3 Firebase projects.
```

```
✓ Select a Firebase project to configure your Flutter application with · flutterfirebaseeducated (FlutterFirebaseEducated)
```

```
? Which platforms should your configuration support (use arrow keys & space to select)? >
```

```
✓ android
```

```
ios
```

```
macos
```

```
web
```

```
windows
```

หากต้องการให้รันทดสอบบน Chrome ได้ด้วย ให้เลือก / web รวมไว้ด้วย (กด spacebar ให้มีเครื่องหมายถูกด้านหน้า)

การตั้งค่าเริ่มต้นในไฟล์ main.dart

- หลังจากตั้งค่าโปรเจกต์ Firebase เรียบร้อยแล้ว จะต้องเริ่มต้น (initialize) Firebase ในแอปพลิเคชัน Flutter ภายในไฟล์ main.dart ดังนี้

```
1  import 'package:flutter/material.dart';
2  import 'package:firebase_core/firebase_core.dart';
3  import 'firebase_options.dart';
4  import 'pages/homepage.dart';
5
   Run | Debug | Profile
6  void main() async {
7    WidgetsFlutterBinding.ensureInitialized();
8    await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
9    runApp(const MyApp());
10 }
```


Data Model Design

- ข้อมูลที่ต้องการจัดเก็บประกอบด้วย
 - รหัสผู้ใช้ไฟ
 - เดือน/ปี ที่ใช้ไฟ
 - จำนวนหน่วยที่ใช้
 - จำนวนเงิน
 - สถานะการชำระเงิน (Paid/Unpaid)

+ Start collection

electricity_bills >

ตัวอย่าง บน Firestore

+ Add document

Kz8JNA3GgiRdl1QLhKd1 >

miBpJK0Uj5wB5ksLjfV0

uxASTm7Td272JM7tsHX8

wtvbPAXhiWxMDpcPC9dz

ตัวอย่าง JSON

```
{
  "electricity_bills" : [
    {
      "userId": "U001",
      "month": "February 2026",
      "units": 150,
      "amount": 620,
      "paidStatus": "Paid",
    }
  ]
}
```

+ Start collection

+ Add field

amount: 620

month: "February 2026"

paidStatus: "Paid"

units: 150

userId: "U001"

Data model Class

- สร้าง class เฉพาะสำหรับใช้งานกับ Data Model ที่เรากำหนดขึ้น
- ตั้งชื่อ class ว่า `BillingRecordModel`
- บันทึกไว้ในไฟล์ชื่อว่า `billing_record_model.dart`

```
1 class BillingRecordModel {
2   String userId;
3   String month;
4   int units;
5   double amount;
6   String paidStatus; // 'Paid' หรือ 'Unpaid'
7   String? referenceId;
8
9   static const CollectionName = 'electricity_bills';
10
11   BillingRecordModel({
12     required this.userId,
13     required this.month,
14     required this.units,
15     required this.amount,
16     required this.paidStatus,
17     this.referenceId,
18   });
19
20   factory BillingRecordModel.fromJson(Map<String, dynamic> json) {
21     return BillingRecordModel(
22       userId: json['userId'] ?? '',
23       month: json['month'] ?? '',
24       units: json['units'] ?? 0,
25       // ใช้ .toDouble() กับ amount เสมอ ไม่ว่าค่าจาก Firestore จะเป็น int หรือ double
26       amount: (json['amount'] as num).toDouble(),
27       paidStatus: json['paidStatus'] ?? 'Unpaid',
28     );
29   }
30
31   Map<String, dynamic> toJson() {
32     return {
33       'userId': userId,
34       'month': month,
35       'units': units,
36       'amount': amount,
37       'paidStatus': paidStatus,
38     };
39   }
40 }
```

สร้างคลาสสำหรับจัดการข้อมูลบน Firebase

- สร้างไฟล์ชื่อว่า **database_helper.dart** ไว้ในโฟลเดอร์ /services เพื่อเขียนคำสั่งสำหรับเข้าถึงข้อมูลบน Firebase

```
1  import '../models/billing_record_model.dart';
2  import 'package:cloud_firestore/cloud_firestore.dart';
3
4  class DatabaseHelper {
5      final CollectionReference collection = FirebaseFirestore.instance.collection(BillingRecordModel.CollectionName);
6
7      // insert a new billing record
8      Future<DocumentReference> addBillingRecord(BillingRecordModel billingRecord) async {
9          | return await collection.add(billingRecord.toJson());
10     }
11     // update an existing billing record
12     Future<void> updateBillingRecord(String id, BillingRecordModel billingRecord) async {
13         | return await collection.doc(id).update(billingRecord.toJson());
14     }
15     // delete a billing record
16     Future<void> deleteBillingRecord(String id) async {
17         | return await collection.doc(id).delete();
18     }
19     // load all billing records
20     Stream<QuerySnapshot> getStreamBillingRecords() {
21         | return collection.snapshots();
22     }
23 }
```

การใช้งาน onChanged กับ onSave ใน Form

คุณสมบัติ	onChanged	onSave
ความถี่ในการทำงาน	ทุกครั้งที่กดปุ่มคีย์บอร์ด	ครั้งเดียวเมื่อสั่ง .save()
Trigger	การกระทำของ User (User Input)	คำสั่งจากโค้ด (Programmatic)
การใช้งานหลัก	UI Interaction / Real-time Logic	Data Collection / Submission
ต้องใช้ GlobalKey หรือไม่	ไม่จำเป็น	จำเป็นต้องมี

การอ่านข้อมูลจาก Firebase เพื่อมาแสดงด้วย ListView

- เรียกใช้งาน StreamBuilder
- การใช้ StreamBuilder จะเชื่อมต่อกับ Firebase แบบ Realtime หากเกิดการเปลี่ยนแปลงบน Cloud ข้อมูลที่แสดงบน Mobile App จะเปลี่ยนแปลงตามโดยอัตโนมัติ

homepage.dart

```
31 body: StreamBuilder(  
32   stream: DatabaseHelper().getStreamBillingRecords(),  
33   builder: (BuildContext context, AsyncSnapshot<QuerySnapshot> snapshot) {  
34     if (!snapshot.hasData) {  
35       return Center(child: CircularProgressIndicator());  
36     }  
37     if (snapshot.data!.docs.isEmpty) {  
38       return Center(child: Text('No billing records found.'));  
39     }  
40     return _buildListView(snapshot);  
41   },  
42   ), // StreamBuilder
```

database_helper.dart

```
19 // load all billing records  
20 Stream<QuerySnapshot> getStreamBillingRecords() {  
21   return collection.snapshots();  
22 }  
23 }
```

แสดง ListView จาก Snapshot

- นำ Snapshot มาถ่ายลง Array ของ BillingRecord
- จากนั้นจึงนำ Array ที่ได้ไปสร้างเป็น ListView

```
74 // Build the ListView for displaying billing records
75 Widget _buildListView(AsyncSnapshot snapshot) {
76   billingItems.clear();
77   for (var doc in snapshot.data!.docs) {
78     billingItems.add(
79       BillingRecordModel(
80         userId: doc.get('userId'),
81         month: doc.get('month'),
82         units: doc.get('units') as int,
83         amount: doc.get('amount') is int
84           ? (doc.get('amount') as int).toDouble()
85           : doc.get('amount') as double,
86         paidStatus: doc.get('paidStatus'),
87         referenceId: doc.id,
88       ),
89     ); // Update the snapshot data with the model
90   }
```

วิธีการนี้ไม่ได้เป็นวิธีเดียวที่สามารถทำได้
เพียงแต่เป็นวิธีที่ตัวอย่างนี้นำมาใช้เท่านั้น
ซึ่งเรายังมีวิธีที่นำ Snapshot
ไปใช้ได้โดยตรงก็สามารถทำได้

แก้ปัญหาเมื่อข้อมูลที่ถูกบันทึกบน
Firestore เป็น int จะเปลี่ยนให้เป็น
double เสมอ ก่อนนำไปแสดง

แสดง ListView จาก Snapshot (cont.)

- นำ Array ที่ได้ไปสร้างเป็น ListView

```
92 return ListView.separated(  
93     itemCount: billingItems.length, ,  
94     itemBuilder: (BuildContext context, int index) {  
95         String titleDate = billingItems[index].month;  
96         String paidStatusText = billingItems[index].paidStatus == 'Paid' ? 'ชำระแล้ว' : 'ยังไม่ชำระ';  
97         String subtitle =  
98             "หน่วยที่ใช้ ${billingItems[index].units} หน่วย, ${billingItems[index].amount} บาท\n$paidStatusText";  
99         return ListTile(  
100             title: Text(titleDate, style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),  
101             subtitle: Text(subtitle, style: TextStyle(fontSize: 14)),  
102             onTap: () {},  
103         ); // ListTile  
104     },  
105     separatorBuilder: (BuildContext context, int index) {  
106         return Divider(color: Colors.grey);  
107     },  
108 ); // ListView.separated
```


การโหลดข้อมูลจาก Firebase

หัวข้อ	StreamBuilder	FutureBuilder
การอัปเดตข้อมูล	อัปเดตแบบเรียลไทม์เมื่อข้อมูลใน Firestore เปลี่ยน	ดึงข้อมูลครั้งเดียวเมื่อเรียก Future
การใช้งาน	ใช้กับ Stream เช่น <code>FirebaseFirestore.instance.collection(...).snapshots()</code>	ใช้กับ Future เช่น <code>FirebaseFirestore.instance.collection(...).get()</code>
เหมาะกับ	ข้อมูลที่เปลี่ยนแปลงบ่อย เช่น แชท, feed, การแจ้งเตือน	ข้อมูลคงที่หรือโหลดครั้งเดียว เช่น รายละเอียดโปรไฟล์, การตั้งค่า
การรีโหลด	ไม่ต้องรีโหลดเอง ระบบจะอัปเดตอัตโนมัติ	ต้องเรียก Future ใหม่เพื่อโหลดข้อมูลอีกครั้ง

การบ้าน #7

- ให้แก้ไขเพิ่มเติมแอปพลิเคชัน Electricity Payment โดยเมื่อต้องการลบข้อมูลรายการใดก็ตาม ให้ใช้วิธี Swipe จากขวาไปซ้าย และจะปรากฏไอคอนให้กดลบได้ และจะต้องผ่านการยืนยันก่อนทุกครั้ง โดยการยืนยันให้ประกอบด้วยตัวเลือก Yes/No สามารถเลือกได้ หลังจากลบแล้วข้อมูลบน Cloud จะต้องถูกลบ และรายการที่แสดงที่หน้า Home จะต้องอัปเดตตามด้วยเช่นกัน
- ให้เพิ่มเติมการแก้ไขโดยเมื่อผู้ใช้ Swipe จากหน้าไปหลัง หรือจากซ้ายไปขวา ให้ปรากฏไอคอน เพื่อให้กดแก้ไขได้ เมื่อกดเลือกให้แสดงฟอร์มสำหรับกรอกข้อมูล โดยแสดงค่าเดิมของรายการที่เลือก เมื่อกด Submit หรือบันทึก ข้อมูลในฟอร์มจะต้องไป update บน Cloud Firestore Database และรายการที่แสดงที่หน้า Home จะต้องอัปเดตตามด้วยเช่นกัน

Q/A